Temporal Checkpoints & Explanations

> **One-line definition:**
> Temporal is a durable execution platform that turns ordinary Python functions into fault-tolerant, stateful, long-running processes — without you writing retry loops, state stores, or recovery logic.

Support Legend

Label	Meaning
Supported	Built into Temporal — zero extra infrastructure
Partially Supported	Supported via a design pattern — requires intentional code structure
Not Supported	Needs an external system; Temporal orchestrates but doesn't provide it

---

Table of Contents

---

1. Loops Inside Temporal

Status: Supported

How Temporal Handles It

- Workflows are **replayed** from their event history when a Worker restarts.
- Every completed Activity inside the loop is recorded as an event; on replay its result is read from history — the Activity is not re-executed.

- Loop control (stop flags, max counts) lives in **Workflow state** — signals flip these flags from outside.
- For history-size safety on very long loops, use `continue_as_new` to carry forward only essential state and start fresh history.

> **Note:** Activity results inside loops are cached in event history — Temporal never re-calls an Activity during replay; it reads the recorded result. This is what makes the loop durable, not magic. History size limit (~50k events) matters for long loops — use `continue_as_new` to avoid hitting the limit in production infinite-loop workflows.

---

2. Retries

Status: Supported

How Temporal Handles It

- The Temporal Cluster records `ActivityTaskFailed` in the event history.
- It schedules a new `ActivityTaskScheduled` after the back-off interval.
- Each retry attempt increments `activity.info().attempt`.
- The Workflow code **sees only the final success** — intermediate failures are invisible to Workflow logic unless you explicitly inspect the attempt count.
- `non_retryable=True` on an `ApplicationError` stops retries immediately — useful for business validation errors.

> **Note:** Default behaviour is unlimited retries with exponential back-off (`backoff_coefficient=2.0`, cap at 100× initial). Always set `maximum_attempts` for Activities that call external services. Retries do not re-enter the Workflow — only the Activity is retried. The Workflow thread stays parked waiting for the Activity result.

---

3. Audit Functionality / Monitoring

Status: Supported (Event History) + **Not Supported** (Metrics/Alerting)

How Temporal Handles It

- **Event history** is the built-in audit trail. Every `ActivityTaskScheduled`, `ActivityTaskCompleted`, `TimerFired`, `SignalReceived`, etc. is recorded with timestamps and payloads.
- The **Temporal Web UI** (`:8233`) visualises the event history graphically.
- The **CLI** lets you inspect it programmatically: `temporal workflow show --workflow-id <id>`
- `workflow.logger` and `activity.logger` emit structured logs that are suppressed during replay (preventing duplicate log entries).
- For external observability (Prometheus metrics, Datadog, PagerDuty), Temporal exports SDK and server metrics that you connect to your existing monitoring stack.

> **Note:** `workflow.logger` is replay-safe — it suppresses duplicate log lines during replay. Never use `print()` or standard `logging` directly inside a Workflow. Heartbeats serve as a sub-activity audit trail — they record progress within a long-running Activity, and if the Worker crashes, the next attempt can pick up from the last heartbeat checkpoint.

4. Trigger Mechanisms

Status: Supported (Signals, Schedules, Child Workflows) + **Not Supported** (HTTP, Events)

How Temporal Handles It

Trigger Type	Mechanism
Code / API call	`client.start_workflow()` or `client.execute_workflow()`
Scheduled (cron)	`CronSchedule` or Temporal Schedules API
Signal from outside	`handle.signal(workflow.some_signal, payload)`
Child workflow	`workflow.start_child_workflow()` from a parent
External event	Your code receives the event and calls `client.start_workflow()`

> **Note:** Signals are not triggers for new workflows — they are messages to a running workflow. To trigger from an external event (Kafka, webhook), your consumer code calls `client.start_workflow()` or `handle.signal()`. Temporal Schedules (the newer API replacing `cron_schedule`) offer pause/unpause, backfill, and jitter — more control than a plain cron.

5. Data Capture

Status: Supported (Event History Payloads) + **Partially Supported** (Custom Storage via Activities)

How Temporal Handles It

- **DataConverter** serialises inputs/outputs (JSON by default) into the event history.
- `dataclass` objects are natively serialised — use typed dataclasses instead of raw dicts for structured capture.
- **Search Attributes** let you tag Workflow executions with indexed custom metadata (e.g., customer ID, order status) that you can query across all executions.
- For long-term persistence outside Temporal (data warehouse, DB), use an Activity that writes to your storage system.

> **Note:** The event history is not a database — it holds execution data, not domain data. For analytical queries or long-term reporting, write to an external store via an Activity. Custom DataConverters let you encrypt payloads at rest — critical for PII — keeping sensitive data from appearing in plain text in the Web UI.

6. Validation and Correctness

Status: Partially Supported

How Temporal Handles It

- **Input validation** is done at the Activity level using `ApplicationError` with `non_retryable=True` — tells Temporal "this is a business logic error, not a transient failure; don't retry."
- **Typed dataclasses** enforce structure at serialisation time through Python's type system.
- **Update validators** (`@workflow.update_validator`) let you reject an update before it modifies Workflow state.
- **Workflow-level guards** check state in the `@workflow.run` method before dispatching Activities.

> **Note:** `non_retryable=True` is the key distinction between a transient failure (retry) and a business error (fail fast). Omitting it on validation errors wastes retries and confuses debugging. Temporal cannot validate data before a Workflow starts — that is the responsibility of the client code or an initial validation Activity.

---

7. Decision and Routing

Status: Supported

How Temporal Handles It

- Conditional logic in Workflow code determines which Activities to execute, in what order, and with what parameters.
- Routing based on **runtime data** (e.g., distance > 25 km → reject order) is handled by reading Activity results and branching.
- **Signals** let external systems inject routing decisions into a running Workflow.
- **Child Workflows** route entire sub-processes to different Task Queues or Workers.

> **Note:** All routing logic must be deterministic — the same inputs must always produce the same branch. Never route based on `random.random()`, `datetime.now()`, or external API calls directly inside the Workflow. Drive conditional logic from Activity results.

---

8. Human Interaction

Status: Partially Supported

How Temporal Handles It

- Workflow calls `await workflow.wait_condition(lambda: self._approved)`.
- This is a **durable wait** — the Worker can restart, the cluster can restart, and the wait survives.
- A frontend/backend hits your API, which sends `handle.signal(workflow.approve, reviewer_id)`.
- The Workflow state flag flips, `wait_condition` resolves, and execution continues.
- **Queries** let the UI poll current status ("Is this still waiting for approval?") without modifying state.

> **Note:** There is no built-in UI for human interaction — you build it. Temporal provides the reliable waiting and waking mechanism; your frontend and API provide the human interface. Set a timeout on `wait_condition` so workflows don't wait forever if a human never acts.

---

9. Execution (External Systems)

Status: Supported (via Activities) + **Not Supported** (The Systems Themselves)

How Temporal Handles It

- The Workflow calls `workflow.execute_activity(call_payment_api, ...)`.
- The Activity runs on a Worker — it has full access to the network, DB drivers, HTTP clients, etc.
- Temporal records the result in the event history on success.
- If the external call fails, Temporal retries the Activity per the `RetryPolicy`.
- **Class-based Activities** support dependency injection (shared HTTP session, DB connection pool) — the right pattern for external system clients.

> **Note:** Activities must be idempotent when calling external systems. Temporal may retry an Activity even after it partially succeeded (e.g., if the Worker crashed before recording the result). Design your external calls to be safe to call twice — use idempotency keys. Use `heartbeat_timeout` on long-running Activity calls so Temporal detects a stuck Worker quickly and retries on another.

---

10. Sequential vs Parallel Execution

Status: Supported

How Temporal Handles It

- **Sequential**: `await workflow.execute_activity(A)` then `await workflow.execute_activity(B)` — B starts only after A completes.
- **Parallel via `asyncio.gather`**: Both activities are scheduled simultaneously; the Workflow resumes when both are done.
- **Parallel via `start_activity`**: Returns a handle immediately; you can launch many activities and await handles later, or never (fire-and-forget within workflow).
- **Child Workflows in parallel**: Start multiple child workflows and `asyncio.gather` their handles.
- All of the above are **fully durable** — Temporal tracks every scheduled/completed event.

> **Note:** `asyncio.gather` inside a Workflow is deterministic because Temporal controls the event loop. Do not use `asyncio.create_task()` or raw threads inside a Workflow — both break determinism. Sync Activities run on a thread pool, not the event loop — they can be truly parallel in CPU time, but from the Workflow's perspective they're just Activities that return results.

---

11. Stateful Workflow

Status: Supported

How Temporal Handles It

- Workflow state lives in Python instance variables (`self.*`).
- On Worker restart, Temporal sends the full event history to a new Worker.
- The Worker replays the Workflow code from scratch — `__init__` runs, then every event is applied in order — restoring all `self.*` fields to their exact pre-crash values.
- **Signals** are the primary way to mutate state from outside.
- **Queries** read state without mutating it (safe to call from anywhere, anytime).

> **Note:** Do not persist state in a database from within the Workflow function itself. Workflow code runs during replay — writing to a DB inside `@workflow.run` would double-write on every replay. All side effects must go through Activities. Workflow history is the state store — you don't need Redis or a DB to remember where you are; Temporal already does that.

---

12. Deterministic Execution (Idempotency)

Status: Supported (Enforced by Design)

How Temporal Handles It

- Temporal runs Workflow code in a **sandboxed event loop** that intercepts non-deterministic operations.
- During replay, Activity calls are **not re-executed** — their recorded results are injected back in.
- `asyncio.sleep()` becomes a durable timer, not a real sleep — it's skipped instantly during replay.
- `workflow.now()` returns the Workflow-safe current time (from event history), not the system clock.
- **Non-determinism errors** (`WorkflowTaskFailed`) occur when a code change between deployments alters the execution path — a critical failure mode to understand.

> **Note:** Determinism is not optional — it is required for correctness. A non-deterministic Workflow will produce a `WorkflowTaskFailed` error during replay due to an event history mismatch. When you need to change Workflow logic that is currently running in production, use the Workflow Versioning API (`workflow.patched()`) to branch behaviour based on whether the execution predates the change.

---

13. Exception Handling

Status: Supported

How Temporal Handles It

Error Type	Class	Behaviour
Transient (network, timeout)	Any `Exception`	Retried per `RetryPolicy`
Business logic error	`ApplicationError(non_retryable=True)`	Fails immediately, no retry
Workflow failure	`ApplicationError` (from Activity exhausted)	Workflow transitions to `Failed`
Workflow cancellation	`CancelledError`	Workflow transitions to `Cancelled`
Workflow termination	External `terminate()` call	Immediate stop, no cleanup

> **Note:** An `ActivityError` wraps the root cause — inspect `.cause` to get the original exception. Raising inside Workflow code (not an Activity) immediately fails the Workflow — no retry. Only

> Activities are automatically retried. Use this intentionally when a workflow-level condition is unrecoverable.

---

14. Versioning

Status: Supported (Patching API) + **Partially Supported** (Requires Code Discipline)

How Temporal Handles It

- `workflow.patched("patch-id")` returns `True` for new executions that know about the patch and `False` for old ones being replayed.
- `workflow.deprecate_patch("patch-id")` removes the old path once all pre-patch executions have completed.
- **Safe deployment order**: Deploy new code with `patched()` → wait for old executions to drain → deploy code with `deprecate_patch()` → wait → remove patching code.

> **Note:** Never change Workflow logic that old running executions will replay without using `patched()`. This is the most common production mistake with Temporal — it causes `WorkflowTaskFailed` non-determinism errors. Patching is only needed when changing the sequence of events (adding/removing/reordering Activities, timers, signals). Changing Activity implementation code (inside the Activity function itself) requires no patching.

---

15. Resilience, Traceability, Reconstruction, Controllability

Status: Supported (All Four)

How Temporal Handles It

Property	Mechanism
Resilience	Event history + automatic replay on Worker restart
Traceability	Immutable event log with full payload visibility
Reconstruction	Deterministic replay reconstructs exact state
Controllability	Signals (mutate state), Queries (read state), Updates (sync call+response), `cancel()`, `terminate()`

- **Resilience**: Workflows survive crashes, network failures, and server restarts automatically by replaying the full event history on the next available Worker.
- **Traceability**: Every step is recorded in the immutable event log — you can see what ran, when, with what data, and what failed.
- **Reconstruction**: A crashed workflow resumes exactly where it left off by replaying its history deterministically.
- **Controllability**: You can pause, resume, cancel, send data into, or query any running workflow from outside at any time using Signals, Queries, Updates, `cancel()`, or `terminate()`.

> **Note:** Queries are read-only and synchronous — they return immediately and never change workflow state. Use them freely for monitoring dashboards. Cancellation is cooperative — the Workflow receives a `CancelledError` and can run compensating Activities before finishing. Termination is hard-kill with no cleanup window.

---

16. Batch Execution

Status: Partially Supported

How Temporal Handles It

- **Fan-out**: A parent Workflow starts one child Workflow (or Activity) per batch item using `asyncio.gather` or `start_child_workflow`.
- **Signal-fed batches**: A long-running Workflow receives item IDs via signals, launches an Activity per item in parallel with `start_activity`, and tracks handles.
- **`continue_as_new`**: For very large batches (>thousands of items), break the batch into chunks and restart with `continue_as_new` to keep history size manageable.
- Temporal Schedules drive periodic batch jobs (nightly, hourly).

> **Note:** Temporal is not a batch framework like Spark or Flink. It excels at durable orchestration of heterogeneous steps where individual items may need retries, human intervention, or downstream API calls — not raw number-crunching over millions of rows. Use `continue_as_new` for truly massive batches to avoid the ~50k event history ceiling.

---

17. Real-time Execution

Status: Partially Supported

How Temporal Handles It

- **Latency sources**: Client → Cluster (gRPC) → Task Queue → Worker → Activity → back. Typical round-trip in a well-tuned setup is 50–200ms for a simple Activity.
- **Updates** provide a tighter synchronous loop: caller gets a result back without polling once the Activity completes.
- **Local Activities** (`workflow.execute_local_activity`) run directly on the Worker without a round-trip to the cluster — useful for very fast, low-latency in-process operations.
- Workers on the same machine as the cluster, or Temporal Cloud with low-latency regions, reduce dispatch overhead.

> **Note:** Temporal is optimised for reliability and correctness over raw speed. For sub-100ms SLAs with millions of requests per second, use a cache or a dedicated low-latency service and have Temporal orchestrate the durable parts of the flow. Local Activities bypass task queue dispatch but lose independent retry scope — they're good for read-only cache checks or CPU-bound transforms, not for calls to external services.

Summary Table

#	Checkpoint	Status
1	Loops Inside Temporal	Supported
2	Retries	Supported
3	Audit Functionality / Monitoring	Supported + Not Supported (Metrics)
4	Trigger Mechanisms	Supported + Not Supported (HTTP/Events)
5	Data Capture	Supported + Partially Supported (Custom Storage)
6	Validation and Correctness	Partially Supported
7	Decision and Routing	Supported
8	Human Interaction	Partially Supported
9	Execution (External Systems)	Supported + Not Supported (The Systems)
10	Sequential vs Parallel Execution	Supported
11	Stateful Workflow	Supported
12	Deterministic Execution (Idempotency)	Supported
13	Exception Handling	Supported
14	Versioning	Supported + Partially Supported
15	Resilience, Traceability, Reconstruction, Controllability	Supported
16	Batch Execution	Partially Supported
17	Real-time Execution	Partially Supported